

NEW-AGE GLOBAL UNIVERSITY FOR LIBERAL EDUCATION

Semester: II

Category: Core

Operating System

Course Code: CS1103

(Theory and Lab)

Course Outline

Unit – I

04 Hrs

Introduction to Operating Systems: Operating Systems –Definition Types- Functions -Abstract view of OS- System Structures – System Calls- Virtual Machines.

Unit – II

08 Hrs

Process Management: Process Concepts, Inter-process communication –Threads –Multithreading, Process Scheduling, First-Come, First-Served (FCFS), Shortest Job First, Round Robin (RR), Priority Scheduling.

Unit – III

06 Hrs

Synchronization and Deadlocks: Synchronization –Semaphores –Monitors, Hardware Synchronization – Deadlocks –Methods for Handling Deadlocks.

Unit – IV

12 Hrs

Memory Management: Memory Management Strategies –Contiguous and Non-Contiguous allocation –Paging –Virtual memory Management –File System Design and Implementation – Mass Storage Structure –Disk Scheduling and Management, FCFS, SSTF, and SCAN -I/O Systems- Overview of System Protection and Security.

Contents

- When is a process created?
- Activity: Display of System daemons
- Process Creation
- When do processes end?
- Process Termination
- Hierarchy of process

When is a process created?

Processes can be created in two ways

- **System initialization:** when the OS starts up, one or more processes are created.
- **Execution of a process creation system call:** something explicitly asks for a new process

System calls can come from

- **User request to create a new process** (system call executed from user shell)
- **Already running processes**
 - User programs
 - System daemons // background program of a computer's operating system, performing tasks without the user's direct interaction.

Courtesy: CS 1550, cs.pitt.edu (originally modified by Ethan L. Miller and Scott A. Brandt)

Activity: Display of System daemons

Note: PID - Process ID

PPID - Parent Process ID

PGID - Process Group ID

SID - Session ID

UID - User ID

EUID - Effective User ID

TTY- Teletypewriter (name of terminal)

Target Portal Group (TPG) ID

```
Ubuntu 24.04 LTS - VMware Workstation 17 Player (Non-commercial use only)
Player
Feb 6 11:34
sangeeta@sangeeta: ~
sangeeta@sangeeta:~$ ps -ajx
```

PPID	PID	PGID	SID	TTY	TPGID	STAT	UID	TIME	COMMAND
0	1	1	1	?	-1	Ss	0	0:01	/sbin/init sp
0	2	0	0	?	-1	S	0	0:00	[kthreadd]
2	3	0	0	?	-1	S	0	0:00	[pool_workque
2	4	0	0	?	-1	I<	0	0:00	[kworker/R-rc
2	5	0	0	?	-1	I<	0	0:00	[kworker/R-rc
2	6	0	0	?	-1	I<	0	0:00	[kworker/R-sl
2	7	0	0	?	-1	I<	0	0:00	[kworker/R-ne
2	8	0	0	?	-1	I	0	0:00	[kworker/0:0-
2	9	0	0	?	-1	I<	0	0:00	[kworker/0:0H
2	10	0	0	?	-1	I	0	0:00	[kworker/u256
2	11	0	0	?	-1	I<	0	0:00	[kworker/R-mm
2	12	0	0	?	-1	I	0	0:00	[kworker/0:1-
2	13	0	0	?	-1	I	0	0:00	[rcu_tasks_kt
2	14	0	0	?	-1	I	0	0:00	[rcu_tasks_ru
2	15	0	0	?	-1	I	0	0:00	[rcu_tasks_tr
2	16	0	0	?	-1	S	0	0:00	[ksoftirqd/0]
2	17	0	0	?	-1	I	0	0:00	[rcu_preempt]
2	18	0	0	?	-1	S	0	0:00	[migration/0]
2	19	0	0	?	-1	S	0	0:00	[idle_inject/
2	20	0	0	?	-1	S	0	0:00	[cpuhp/0]
2	21	0	0	?	-1	S	0	0:00	[cpuhp/1]
2	22	0	0	?	-1	S	0	0:00	[idle_inject/
2	23	0	0	?	-1	S	0	0:00	[migration/1]
2	24	0	0	?	-1	S	0	0:00	[ksoftirqd/1]
2	26	0	0	?	-1	I<	0	0:00	[kworker/1:0H
2	27	0	0	?	-1	I	0	0:00	[kworker/u257
2	29	0	0	?	-1	S	0	0:00	[kdevtmpfs]
2	30	0	0	?	-1	I<	0	0:00	[kworker/R-in
2	32	0	0	?	-1	S	0	0:00	[kauditd]
2	34	0	0	?	-1	S	0	0:00	[khungtaskd]
2	35	0	0	?	-1	S	0	0:00	[oom_reaper]
2	37	0	0	?	-1	I<	0	0:00	[kworker/R-wr
2	38	0	0	?	-1	S	0	0:00	[kcompactd0]
2	39	0	0	?	-1	SN	0	0:00	[ksmd]
2	40	0	0	?	-1	I	0	0:00	[kworker/1:1-
2	41	0	0	?	-1	I	0	0:00	[kworker/1:2-
2	42	0	0	?	-1	SN	0	0:00	[khugepaged]
2	43	0	0	?	-1	I<	0	0:00	[kworker/R-ki
2	44	0	0	?	-1	I<	0	0:00	[kworker/R-kb
2	45	0	0	?	-1	I<	0	0:00	[kworker/R-bl
2	46	0	0	?	-1	S	0	0:00	[irq/9-acpi]

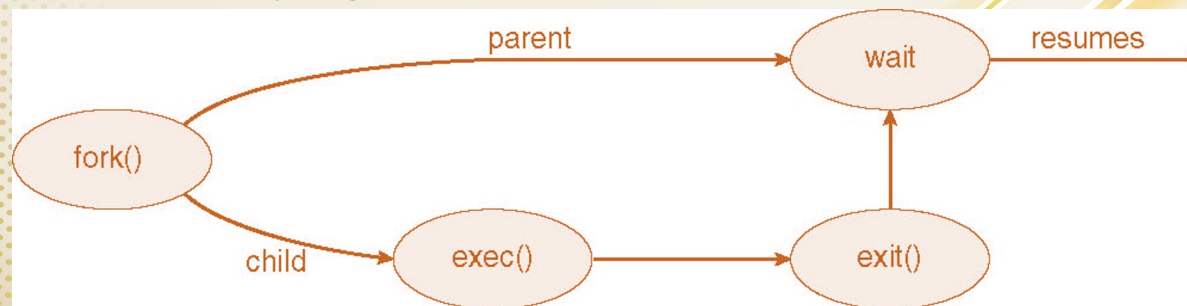
Process Creation

- **Parent** process creates **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a process **identifier (pid)**
- Resource sharing options
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- Execution options
 - Parent and children execute concurrently
 - Parent waits until children terminate

Courtesy: CS/COE 1550 – Operating Systems – Sherif Khattab

Process Creation

- Address space
 - Child duplicate of parent
 - Child has a program loaded into it
- UNIX examples
 - **fork()** system call creates new process
 - **exec()** system call used after a **fork()** to replace the process' memory space with a new program



When do processes end?

Conditions that terminate processes can be

- Voluntary
- Involuntary

Voluntary

- Normal exit
- Error exit

Involuntary

- Fatal error (only sort of involuntary)
- Killed by another process

Courtesy: CS 1550, cs.pitt.edu (originally modified by Ethan L. Miller and Scott A. Brandt)

When do processes end?

- Some operating systems do not allow a child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
 - **cascading termination.** All children, grandchildren, etc. are terminated.
 - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **wait()** system call. The call returns 'status information' and the pid of the terminated process

```
pid = wait(&status);
```

- If no parent waiting (did not invoke **wait()**) process is a **zombie**
- If parent terminated without invoking **wait**, process is an **orphan**

Process Termination

- Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
 - Returns status data from child to parent (via **wait()**)
 - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **abort()** system call. Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates

Hierarchies of Process

Parent creates a child process

- Child processes can create their own children

Forms a hierarchy

- UNIX calls this a “process group”
- If a process exits, its children are “inherited” by the exiting process’s parent

Windows has no concept of process hierarchy

- All processes are created equal

Activity

Execute a program that calls a daemonize function to daemonize itself as a daemon.

Refer to: <https://notes.shichao.io/apue/ch13/>

References

- Silberschatz, Galvin, Gagne, Operating System Concepts, John Wiley and Sons, 10th edition, 2023, ISBN: 935746056X
- Modern Operating Systems by Tanenbaum and Bos , 4th edition, Pearson Publisher, ISBN-13: 978-0-13-359162-0
- Courtesy: Prof. Mouli Sankaran, RV University.